

**Міністерство освіти і науки України**  
**Національний технічний університет України «Київський політехнічний**  
**інститут імені Ігоря Сікорського»**  
**Факультет інформатики та обчислювальної техніки**

**Кафедра інформатики та програмної інженерії**

**Звіт**

з лабораторної роботи № 4 дисципліни  
«Проектування алгоритмів»

**„Проектування структур даних”**

**Виконав(ла)**

ІІ-341 Черепов Олександр Павлович

(шифр, прізвище, ім'я, по батькові)

**Перевірів**

Головченко М.М.

(прізвище, ім'я, по батькові)

Київ 2025

Зміст

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>МЕТА ЛАБОРАТОРНОЇ РОБОТИ.....</b>         | <b>3</b>  |
| <b>2</b> | <b>ЗАВДАННЯ .....</b>                        | <b>4</b>  |
| <b>3</b> | <b>ВИКОНАННЯ .....</b>                       | <b>7</b>  |
| 3.1      | ПСЕВДОКОД АЛГОРИТМІВ.....                    | 7         |
| 3.2      | ЧАСОВА СКЛАДНІСТЬ ПОШУКУ.....                | 10        |
| 3.3      | ПРОГРАМНА РЕАЛІЗАЦІЯ.....                    | 10        |
| 3.3.1    | <i>Вихідний код.....</i>                     | <i>11</i> |
| 3.3.2    | <i>Приклади роботи .....</i>                 | <i>31</i> |
| 3.4      | ТЕСТУВАННЯ АЛГОРИТМУ .....                   | 32        |
| 3.4.1    | <i>Часові характеристики оцінювання.....</i> | <i>32</i> |
|          | <b>ВИСНОВОК.....</b>                         | <b>33</b> |
|          | <b>КРИТЕРІЇ ОЦІНЮВАННЯ.....</b>              | <b>34</b> |

## 1 МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Мета роботи – вивчити основні підходи проєктування та обробки складних структур даних.

## 2 ЗАВДАННЯ

Відповідно до варіанту (таблиця 2.1), записати алгоритми пошуку, додавання, видалення і редагування запису в структурі даних за допомогою псевдокоду (чи іншого способу по вибору).

Записати часову складність пошуку в структурі в асимптотичних оцінках.

Виконати програмну реалізацію невеликої СУБД з графічним (не консольним) інтерфейсом користувача, з наочним відображенням структури даних (до 100 елементів чи більше) (дані БД мають зберігатися на ПЗП), з функціями пошуку (алгоритм пошуку у вузлі чи блоці структури згідно варіанту таблиця 2.1, за необхідності), додавання, видалення та редагування записів (запис складається із ключа і даних, ключі унікальні і цілочисельні, даних може бути декілька полів для одного ключа, але достатньо одного рядка фіксованої довжини). Для зберігання даних використовувати структуру даних згідно варіанту (таблиця 2.1).

Заповнити базу випадковими значеннями до 10000 і зафіксувати середнє (із 20-25 пошуків) число порівнянь для знаходження запису по ключу. За допомогою одного чи декількох, чат-ботів отримати рекомендації з покращення програми та відобразити їх у висновку.

Зробити висновок з лабораторної роботи.

Таблиця 2.1 – Варіанти алгоритмів

| № | Структура даних                                                             |
|---|-----------------------------------------------------------------------------|
| 1 | Файли з щільним індексом з перебудовою індексної області, бінарний пошук    |
| 2 | Файли з щільним індексом з областю переповнення, бінарний пошук             |
| 3 | Файли з не щільним індексом з перебудовою індексної області, бінарний пошук |
| 4 | Файли з не щільним індексом з областю переповнення, бінарний пошук          |
| 5 | АВЛ-дерево                                                                  |

|    |                                                                                        |
|----|----------------------------------------------------------------------------------------|
| 6  | Червоно-чорне дерево                                                                   |
| 7  | В-дерево $t=10$ , бінарний пошук                                                       |
| 8  | В-дерево $t=25$ , бінарний пошук                                                       |
| 9  | В-дерево $t=50$ , бінарний пошук                                                       |
| 10 | В-дерево $t=100$ , бінарний пошук                                                      |
| 11 | Файли з щільним індексом з перебудовою індексної області, однорідний бінарний пошук    |
| 12 | Файли з щільним індексом з областю переповнення, однорідний бінарний пошук             |
| 13 | Файли з не щільним індексом з перебудовою індексної області, однорідний бінарний пошук |
| 14 | Файли з не щільним індексом з областю переповнення, однорідний бінарний пошук          |
| 15 | АВЛ-дерево                                                                             |
| 16 | Червоно-чорне дерево                                                                   |
| 17 | В-дерево $t=10$ , однорідний бінарний пошук                                            |
| 18 | В-дерево $t=25$ , однорідний бінарний пошук                                            |
| 19 | В-дерево $t=50$ , однорідний бінарний пошук                                            |
| 20 | В-дерево $t=100$ , однорідний бінарний пошук                                           |
| 21 | Файли з щільним індексом з перебудовою індексної області, метод Шарра                  |
| 22 | Файли з щільним індексом з областю переповнення, метод Шарра                           |
| 23 | Файли з не щільним індексом з перебудовою індексної області, метод Шарра               |
| 24 | Файли з не щільним індексом з областю переповнення, метод Шарра                        |
| 25 | АВЛ-дерево                                                                             |
| 26 | Червоно-чорне дерево                                                                   |
| 27 | В-дерево $t=10$ , метод Шарра                                                          |
| 28 | В-дерево $t=25$ , метод Шарра                                                          |

|    |                                              |
|----|----------------------------------------------|
| 29 | В-дерево $t=50$ , метод Шарра                |
| 30 | В-дерево $t=100$ , метод Шарра               |
| 31 | АВЛ-дерево                                   |
| 32 | Червоно-чорне дерево                         |
| 33 | В-дерево $t=250$ , бінарний пошук            |
| 34 | В-дерево $t=250$ , однорідний бінарний пошук |
| 35 | В-дерево $t=250$ , метод Шарра               |

## 3.1 Псевдокод алгоритмів

***Процедура пошуку запису:***

```

procedure Search(key)
  if key <= MAIN_INDEX_CAPACITY then
    blockIndex = (key - 1) // BLOCK_CAPACITY
  else
    blockIndex = BLOCKS_COUNT - 1
  end if

  indices = GetIndexBlock(blockIndex)

  if length(indices) == 0 then
    return Null
  end if

  minIndex = indices[0].key
  maxIndex = indices[last].key

  outputData = Null

  if minIndex <= key <= maxIndex then
    pos = InterpolationSearch(indices.keys, key)

    if pos != -1 then
      record = indices[pos]
      outputData = ReadDataFromFile(record.number)
      area = "main"
    end if
  end if

  if outputData is Null then
    overflowIndices = GetOverflowIndices()

    for i = 0 to length(overflowIndices) - 1 do
      record = overflowIndices[i]
      if record.key == key then
        outputData = ReadDataFromFile(record.number)
        area = "overflow"
        break
      end if
    end for
  end if

  if outputData is not Null then
    return { "data": outputData, "area": area }
  else
    return Null
  end if
end procedure

```

### ***Алгоритм пошуку:***

```
procedure InterpolationSearch(keys, targetKey)
    low = 0
    high = length(keys) - 1

    while low <= high and keys[low] <= targetKey <= keys[high] do
        if high == low then
            if keys[low] == targetKey then
                return low
            end if
            break
        end if

        pos = low + ((targetKey - keys[low]) * (high - low)) // (keys[high] -
keys[low])

        if keys[pos] == targetKey then
            return pos
        end if

        if keys[pos] < targetKey then
            low = pos + 1
        else
            high = pos - 1
        end if
    end while

    return -1
end procedure
```

### ***Процедура додавання запису:***

```
procedure AddRecord(data, key)
    fileSize = GetFileSize(DataPath)
    recordNumber = fileSize // (RECORD_SIZE + 1)

    if key == -1 then
        key = DefineAutoKey()
    end if

    if IndexExists(key) then
        return Error("Duplicate Key")
    end if

    AppendToFile(DataPath, data)

    if key > MAIN_INDEX_CAPACITY then
        blockIndex = BLOCKS_COUNT - 1
    else
        blockIndex = (key - 1) // BLOCK_CAPACITY
    end if

    indices = GetIndexBlock(blockIndex)

    if length(indices) < BLOCK_CAPACITY then
```



```

        insertPos = 0
        while insertPos < length(indices) and indices[insertPos].key < key do
            insertPos = insertPos + 1
        end while

        InsertToArray(indices, insertPos, {key, recordNumber})

        RewriteBlock(blockIndex, indices)
    else
        AppendToFile(OverflowPath, {key, recordNumber})
    end if

    return key
end procedure

```

### ***Процедура редагування запису:***

```

procedure UpdateRecord(key, newValue)
    recordInfo = Search(key)

    if recordInfo is Null then
        return Error("Not Found")
    end if

    formattedValue = FormatToFixedSize(newValue, RECORD_SIZE) + "\n"
    fileData = ReadAllLines(DataPath)

    fileData[recordInfo.number] = formattedValue

    OverwriteAllLines(DataPath, fileData)
    return Success
end procedure

```

### ***Процедура видалення запису:***

```

procedure DeleteRecord(key)
    recordToDelete = Search(key)

    if recordToDelete is Null then
        return Null
    end if

    fileData = ReadAllLines(DataPath)
    RemoveAt(fileData, recordToDelete.number)
    OverwriteFile(DataPath, fileData)

    if recordToDelete.area == "overflow" then
        overflowData = ReadAllLines(OverflowPath)
        RemoveAt(overflowData, recordToDelete.indexPos)
        OverwriteFile(OverflowPath, overflowData)
    else
        blockIndex = (key - 1) // BLOCK_CAPACITY
        indices = GetIndexBlock(blockIndex)

        RemoveFromList(indices, key)
    end if
end procedure

```

```

        RewriteBlock(blockIndex, indices)
    end if

    return recordToDelete
end procedure

```

### 3.2 Часова складність пошуку

Часова складність процедури пошуку залежить від розподілу даних між основною областю та областю переповнення.

***Пошук в основній області (метод Шарра):***

- Середня складність:  $O(\log(\log N))$ , де  $N$  – це кількість записів в основній області.
- Найгірший випадок:  $O(N)$ . Це трапляється при сильно нерівномірному розподілі ключів, коли інтерполяція постійно помиляється на невелику величину.

***Пошук в області переповнення (лінійний пошук):***

- Складність:  $O(M)$ , де  $M$  – кількість записів в області переповнення. Оскільки дані в цій області не впорядковані, необхідно послідовно перевіряти кожен елемент до знаходження потрібного.

***Загальна складність алгоритму:***

- У разі успішного пошуку в основній області:  $O(\log(\log N))$ ;
- У разі необхідності перевірки області переповнення:  $O(\log(\log N) + M)$ .

### 3.3 Програмна реалізація

Структура проекту та файлів даних:

```

programme/
├─ data/
│   ├─ data.dat
│   ├─ index.dat
│   └─ overflow.dat
├─ src/
└─ app/

```

```

|   |   └─ app.py
|   |   └─ initializer.py
|   └─ storage/
|       └─ manager.py
|   └─ ui/
|       └─ main_window.py
|   └─ utils/
|       └─ logger.py
|       └─ validator.py
|   └─ main.py
└─ .gitignore

```

### 3.3.1 Вихідний код

#### ***app.py:***

```

from utils.logger import Logger
from app.initializer import Initializer
from storage.manager import StorageManager

class App:
    def __init__(self, data_dir_path, record_size, index_record_size,
main_index_capacity, block_capacity):
        self.data_dir = data_dir_path

        self.initializer = Initializer(self.data_dir, index_record_size,
block_capacity, main_index_capacity)
        self.storage = StorageManager(self.data_dir, record_size,
index_record_size, main_index_capacity, block_capacity)
        self.logger = Logger("App")

        self.BLOCK_CAPACITY = block_capacity
        self.MAIN_INDEX_CAPACITY = main_index_capacity
        self.PAGE_LIMIT = 100

    def start(self):
        self.logger.info("--- BEGINNING WORKSPACE INITIALIZATION ---")

        if self.BLOCK_CAPACITY > self.MAIN_INDEX_CAPACITY:
            self.logger.error_and_exit("Block capacity can't be larger than main
index capacity", 1)

        if self.MAIN_INDEX_CAPACITY % self.BLOCK_CAPACITY != 0:
            self.logger.error_and_exit("Block capacity and main index capacity
values must be multiples", 1)

```

```

        try:
            self.initializer.init_workspace()
        except Exception as e:
            self.logger.error_and_exit(f"An unexpected error occurred: {e}", 1)

        self.logger.info("--- WORKSPACE INITIALIZATIZED SUCCESSFULLY ---")

    def get_records(self, page):
        self.logger.info(f"--- GETTING RECORDS (PAGE {page + 1}) ---")

        try:
            all_indices = self.storage.get_all_records()
            all_indices.sort(key=lambda x: x[0])

            total_records = len(all_indices)
            total_pages = (total_records + self.PAGE_LIMIT - 1) //
self.PAGE_LIMIT

            if total_pages == 0: total_pages = 1
            if page >= total_pages: page = total_pages - 1
            if page < 0: page = 0

            start = page * self.PAGE_LIMIT
            end = start + self.PAGE_LIMIT

            page_indices = all_indices[start : end]
            records = []

            for key, _ in page_indices:
                rec = self.storage.search(key)
                if rec:
                    records.append(rec)

            self.logger.info(f"--- RECORDS RECEIVED SUCCSESSFULLY (PAGE
{page+1}/{total_pages}) ---")

            return records, total_pages
        except Exception as e:
            self.logger.error(f"An error occurred while fetching records:
{e}")

            return [], 1

    def add(self, data, key=-1):
        self.logger.info("--- ADDING A NEW RECORD ---")

        try:
            new_id = self.storage.add_record(data, key)
        except Exception as e:
            self.logger.error_and_exit(f"An unexpected error occurred: {e}", 1)

```

```

        if new_id != -1:
            self.logger.info("--- RECORD ADDED SUCCESSFULLY ---")
            return new_id
        else:
            self.logger.info("--- FAILED TO ADD A RECORD ---")
            return -1

def search(self, key):
    self.logger.info("--- SEARCHING FOR A RECORD ---")

    try:
        result = self.storage.search(key)
    except Exception as e:
        self.logger.error_and_exit(f"An unexpected error occurred: {e}", 1)

    if result is not None:
        self.logger.info(f"--- RECORD FOUND SUCCESSFULLY (comparisons:
{result['comparisons']}) ---")
        return result
    else:
        self.logger.info(f"--- FAILED TO FIND A RECORD ---")
        return -1

def update(self, key, new_value):
    self.logger.info("--- UPDATING A RECORD ---")

    try:
        updated = self.storage.update_record(key, new_value)
    except Exception as e:
        self.logger.error_and_exit(f"An unexpected error occurred: {e}", 1)

    if updated is not None:
        self.logger.info("--- RECORD UPDATED SUCCESSFULLY ---")
        return updated
    else:
        self.logger.info("--- FAILED TO UPDATE A RECORD ---")
        return -1

def remove(self, key):
    self.logger.info("--- REMOVING A RECORD ---")

    try:
        updated = self.storage.delete_record(key)
    except Exception as e:
        self.logger.error_and_exit(f"An unexpected error occurred: {e}", 1)

    if updated is not None:
        self.logger.info("--- RECORD REMOVED SUCCESSFULLY ---")
        return updated
    else:
        self.logger.info("--- FAILED TO REMOVE A RECORD ---")

```

```
return -1
```

### ***initializer.py:***

```
import os
from utils.logger import Logger

class Initializer:
    def __init__(self, data_dir_path, index_record_size, block_capacity,
main_index_capacity):
        self.data_dir = data_dir_path

        self.INDEX_RECORD_SIZE = index_record_size
        self.BLOCK_CAPACITY = block_capacity
        self.MAIN_INDEX_CAPACITY = main_index_capacity

        self.BLOCK_SIZE_BYTES = (self.INDEX_RECORD_SIZE + 1) * self.BLOCK_CAPACITY
        self.MAIN_INDEX_SIZE_BYTES = self.BLOCK_SIZE_BYTES *
self.MAIN_INDEX_CAPACITY

        self.data_path = os.path.join(self.data_dir, "data.dat")
        self.index_path = os.path.join(self.data_dir, "index.dat")
        self.overflow_path = os.path.join(self.data_dir, "overflow.dat")

        self.logger = Logger("Initializer")

    def init_workspace(self):
        files_to_check = [self.index_path, self.data_path, self.overflow_path]
        missing_files = []

        self.logger.info("Checking data directory availability...")
        if not os.path.exists(self.data_dir):
            self.logger.info("Data directory not found. Creating from scratch...")

            os.mkdir(self.data_dir)
            self.logger.info("Data directory created successfully")

            self._create_data_files(files_to_check)

        else:
            self.logger.info("Data directory found. Looking for files...")

            for file in files_to_check:
                filename = os.path.basename(file)
                self.logger.info(f"Looking for {filename} file...")

                if os.path.isfile(file):
                    self.logger.info(f"File {filename} found")
```

```

        else:
            self.logger.warning(f"WARNING: File {filename} not found.
Marking as missing...")
            missing_files.append(file)

        if len(missing_files) > 0:
            self.logger.info(f"Creating missing files...
({len(missing_files)})")
            self._create_data_files(missing_files)

        self.logger.info("Workspace initialized successfully!")

    def _create_data_files(self, settings):
        try:
            for file in settings:
                if file == self.index_path:
                    self._create_main_index_file()
                else:
                    with open(file, 'x') as f:
                        self.logger.info(f"File {os.path.basename(file)} created
successfully")
            except Exception as e:
                self.logger.error_and_exit(f"An unexpected error occurred: {e}", 1)

    def _create_main_index_file(self):
        with open(self.index_path, "wb") as f:
            entry_size = self.INDEX_RECORD_SIZE
            capacity = self.MAIN_INDEX_CAPACITY

            empty_record = ((" " * entry_size) + "\n").encode('ascii')

            for _ in range(capacity):
                f.write(empty_record)

            self.logger.info("File index.dat created and filled successfully!")

```

### ***manager.py:***

```

import os
from utils.logger import Logger

class StorageManager:
    def __init__(self, data_dir_path, record_size, index_record_size,
main_index_capacity, block_capacity):
        self.data_dir = data_dir_path
        self.RECORD_SIZE = record_size
        self.INDEX_RECORD_SIZE = index_record_size
        self.BLOCK_CAPACITY = block_capacity

```

```

self.MAIN_INDEX_CAPACITY = main_index_capacity

self.BLOCK_SIZE_BYTES = (self.INDEX_RECORD_SIZE + 1) * self.BLOCK_CAPACITY
self.BLOCKS_COUNT = self.MAIN_INDEX_CAPACITY // self.BLOCK_CAPACITY

self.data_path = os.path.join(self.data_dir, "data.dat")
self.index_path = os.path.join(self.data_dir, "index.dat")
self.overflow_path = os.path.join(self.data_dir, "overflow.dat")

self.logger = Logger("StorageManager")

def add_record(self, data, key=-1):
    self.logger.info(f"Beginning record writing process... (data: {data})")

    if os.path.exists(self.data_path):
        file_size = os.path.getsize(self.data_path)
        record_number = file_size // (self.RECORD_SIZE + 1)
    else:
        record_number = 0

    self.logger.info(f"Calculated record number: {record_number}")\

    new_index = key if key != -1 else self._define_auto_key()

    self.logger.info(f"New index for data: {new_index}")

    if not self._index_exists(new_index, area='main') and not
self._index_exists(new_index, area='overflow'):
        formatted_data = (data.ljust(self.RECORD_SIZE)[:self.RECORD_SIZE] +
"\n").encode('ascii')

        self.logger.info("Writing data...")
        with open(self.data_path, "ab") as f:
            f.write(formatted_data)

        self.logger.info(f"Record data written successfully! ({data})")

        self._write_index(new_index, record_number)

        return new_index
    else:
        self.logger.warning(f"An element with index {new_index} already
exists!")
        return -1

def _define_auto_key(self):
    self.logger.info("Defining auto key...")

    candidate_key = 1
    current_block_idx = 0

```



```

while True:
    if candidate_key > self.MAIN_INDEX_CAPACITY:
        break

    indices = self._get_index_block(current_block_idx)

    existing_keys_in_block = [k for k, v in indices]

    start_key_for_block = (current_block_idx * self.BLOCK_CAPACITY) + 1
    end_key_for_block = start_key_for_block + self.BLOCK_CAPACITY - 1

    for k in range(start_key_for_block, end_key_for_block + 1):
        if k not in existing_keys_in_block:
            if not self._index_exists(k, area='overflow'):
                return k

    current_block_idx += 1
    candidate_key = (current_block_idx * self.BLOCK_CAPACITY) + 1

    candidate_key = self.MAIN_INDEX_CAPACITY + 1
    while self._index_exists(candidate_key, area='overflow'):
        candidate_key += 1

    return candidate_key

def search(self, key):
    self.logger.info(f"Beginning searching process... (key: {key})")
    indices = []
    index_pos = None
    area = None
    output_data = None

    c = 0

    block_index = (key - 1) // self.BLOCK_CAPACITY if key <=
self.MAIN_INDEX_CAPACITY else self.BLOCKS_COUNT-1

    if block_index != -1:
        indices = self._get_index_block(block_index)

    if len(indices) == 0:
        raise Exception("Index file is empty!")

    min_index_main = indices[0][0]
    max_index_main = indices[-1][0]

    c += 2
    if indices and min_index_main <= key <= max_index_main:
        self.logger.info("Searching in the main area...")
        output_data, index_pos, c_main = self._search_main(key, indices)

```

```

        c += c_main

        if output_data is None: self.logger.info("Failed to find in the
main area")

        else: area = f"main [{block_index}]"

    if output_data is None:
        self.logger.info("Searching in the overflow area...")
        indices = self._get_overflow_indices()

        if len(indices) == 0:
            self.logger.warning("Overflow file is empty!")
        else:
            output_data, index_pos, c_overflow =
self._overflow_search(key, indices)

            c += c_overflow

        if output_data:
            area = "overflow"

    if output_data is None:
        self.logger.warning(f"Failed to find a record! (key: {key})")
        return None
    else:
        if os.path.exists(self.data_path):
            record_number = output_data[1]
            with open (self.data_path, "rb") as f:
                for line_no, line in enumerate(f):
                    if line_no == record_number:
                        self.logger.info(f"Record found successfully!
({output_data})")

                        return {
                            "key": key,
                            "number": record_number,
                            "value": line.decode('ascii').strip(),
                            "area": area,
                            "index_pos": index_pos,
                            "comparisons": c
                        }
        else:
            raise Exception("Data file doesn't exist!")

def get_all_records(self):
    self.logger.info(f"Getting all records...")

    all_indices = []

    file_size = os.path.getsize(self.index_path)
    total_blocks = file_size // self.BLOCK_SIZE_BYTES

```

```

    for i in range(total_blocks):
        block = self._get_index_block(i)
        all_indices.extend(block)

    overflow_indices = self._get_overflow_indices()
    all_indices.extend(overflow_indices)

    return all_indices

def update_record(self, key, new_value):
    self.logger.info(f"Beginning data changing process... (key: {key})")
    record_to_change = self.search(key)

    if record_to_change is None:
        return None

    new_data = record_to_change
    new_data["value"] = new_value

    formatted_value = (new_value.ljust(self.RECORD_SIZE)[:self.RECORD_SIZE] +
"\n").encode('ascii')
    file_data = []

    with open (self.data_path, "rb") as f:
        self.logger.info(f"Reading old file data... (area:
{record_to_change['area']}))")
        file_data = f.readlines()

        if len(file_data) < 0:
            raise Exception("Data file is empty!")

        self.logger.info(f"Changing record data... (record_number:
{record_to_change['number']}))")
        file_data[record_to_change["number"]] = formatted_value

    with open(self.data_path, "wb") as f:
        self.logger.info(f"Replacing with updated data... ({new_data})")
        f.writelines(file_data)

    self.logger.info(f"Record updated successfully! (key: {key}, new_value:
{new_value})")
    return new_data

def delete_record(self, key):
    self.logger.info(f"Beginning data deleting process... (key: {key})")
    record_to_delete = self.search(key)

    if record_to_delete is None:
        return None

```

```

        self.logger.info("Deleting a record from data file...")
        self._delete_from_file(self.data_path, record_to_delete["number"])

        self.logger.info(f"Deleting a record from index file... (area:
{record_to_delete['area']}))")
        if record_to_delete["area"] == "overflow":
            self._delete_from_file(self.overflow_path,
record_to_delete["index_pos"])
        else:
            self._delete_index(key)

        return record_to_delete

def _get_index_block(self, block_index):
    self.logger.info(f"Getting indices... (block_index: {block_index})")
    indices = []

    if not os.path.exists(self.index_path):
        return indices

    with open(self.index_path, "rb") as f:
        offset = block_index * self.BLOCK_SIZE_BYTES
        f.seek(offset)
        block_data = f.read(self.BLOCK_SIZE_BYTES)

        for i in range(0, len(block_data), self.INDEX_RECORD_SIZE+1):
            chunk = block_data[i : i + self.INDEX_RECORD_SIZE+1]
            record = self._parse_index_chunk(chunk)

            if record: indices.append(record)

    return indices

def _get_overflow_indices(self):
    self.logger.info(f"Getting indices from overflow area...")
    indices = []

    if os.path.exists(self.overflow_path):
        with open(self.overflow_path, "rb") as f:
            for line in f.readlines():
                record = self._parse_index_chunk(line)
                if record: indices.append(record)

    return indices

def _parse_index_chunk(self, chunk):
    try:
        line = chunk.decode('ascii').strip()
        if not line: return None
        k, v = line.split(',')
        return int(k), int(v)

```

```

        except:
            return None

    def _index_exists(self, index, area='main'):
        self.logger.info(f"Checking if index exists... (area: {area})")

        indices = self._get_index_block((index - 1) // self.BLOCK_CAPACITY) if
area == 'main' else self._get_overflow_indices()
        keys = []
        result = False

        for k, v in indices:
            keys.append(k)

        if int(index) in keys: result = True

        return result

    def _search_main(self, key, indices):
        keys = [k for k, v in indices]

        pos, c_main = self._interpolation_search(keys, target_key=key)

        if pos != -1:
            return indices[pos], pos, c_main
        else:
            return None, None, c_main

    def _overflow_search(self, key, indices):
        c = 0
        for i, record in enumerate(indices):
            c += 1
            if record[0] == key:
                return record, i, c

        return None, None, c

    def _interpolation_search(self, keys, target_key):
        self.logger.info("Using interpolation search...")
        low = 0
        high = len(keys) - 1
        c = 0

        while low <= high and keys[low] <= target_key <= keys[high]:
            c += 1
            if high == low:
                c += 1
                if keys[low] == target_key: return low, c
                break

```

```

        pos = low + ((target_key-keys[low]) * (high-low))//((keys[high]-
keys[low]))

        c += 1
        if keys[pos] == target_key:
            return pos, c

        if keys[pos] < target_key:
            low = pos + 1
        else:
            high = pos - 1

    return -1, c

def _write_index(self, new_idnex, record_number):
    self.logger.info(f"Beginning index writing process...
({new_idnex},{record_number})")

    if new_idnex > self.MAIN_INDEX_CAPACITY:
        block_index = (self.BLOCKS_COUNT) - 1
    else:
        block_index = (new_idnex - 1) // self.BLOCK_CAPACITY

    indices = self._get_index_block(block_index)

    self.logger.info("Defining storage area...")
    if len(indices) < self.BLOCK_CAPACITY:
        self.logger.info("Main area is free. Defining position...")
        insert_pos = 0

        while insert_pos < len(indices) and indices[insert_pos][0] <
new_idnex:
            insert_pos += 1

        self.logger.info(f"Position found. Inserting at [{insert_pos}]...")
        indices.insert(insert_pos, (new_idnex, record_number))

        self.logger.info("Rewriting index file...")
        with open(self.index_path, "r+b") as f:
            offset = block_index * self.BLOCK_SIZE_BYTES

            f.seek(offset)

            for k, v in indices:
                index_data = f"{k},{v}"
                formatted_index_data =
(index_data.ljust(self.INDEX_RECORD_SIZE)[:self.INDEX_RECORD_SIZE] +
"\n").encode('ascii')
                f.write(formatted_index_data)

            empty_slots = self.BLOCK_CAPACITY - len(indices)

```

```

        empty_record = ((" " * self.INDEX_RECORD_SIZE) +
"\n").encode('ascii')

        for _ in range(empty_slots):
            f.write(empty_record)
    else:
        self.logger.info("Main area is full. Writing into overflow...")
        with open(self.overflow_path, "a") as f:
            f.write(f"{new_idnex},{record_number}\n")

        self.logger.info(f"Index data written successfully!
({new_idnex},{record_number})")

def _delete_index(self, index):
    block_index = (index - 1) // self.BLOCK_CAPACITY
    indices = self._get_index_block(block_index)

    for k, v in indices:
        if k == index:
            indices.remove((k, v))
            break

    with open (self.index_path, "r+b") as f:
        offset = block_index * self.BLOCK_SIZE_BYTES

        f.seek(offset)

        for k, v in indices:
            index_data = f"{k},{v}"
            formatted_index_data =
(index_data.ljust(self.INDEX_RECORD_SIZE)[:self.INDEX_RECORD_SIZE] +
"\n").encode('ascii')
            f.write(formatted_index_data)

        empty_record = ((" " * self.INDEX_RECORD_SIZE) + "\n").encode('ascii')
        f.write(empty_record)

def _read_block(self, block_index):
    offset = block_index * self.BLOCK_SIZE_BYTES

    with open (self.index_path, "rb") as f:
        f.seek(offset)
        block_data = f.read(self.BLOCK_SIZE_BYTES)

    return block_data

def _delete_from_file(self, file_path, record_index):
    with open (file_path, "rb") as f:
        self.logger.info("Reading old file data...")
        file_data = f.readlines()

```

```

        if len(file_data) < 0:
            raise Exception("File is empty!")

        self.logger.info(f"Removing a record... (record_number:
{record_index})")
        file_data.pop(record_index)

        with open(file_path, "wb") as f:
            self.logger.info(f"Replacing with updated data...")
            f.writelines(file_data)

```

### **main\_window.py:**

```

import tkinter as tk
from tkinter import ttk, messagebox, simpledialog, Toplevel, Label, Entry, Button,
X
from utils.validator import Validator

class MainWindow(tk.Tk):
    def __init__(self, app, record_size, index_record_size):
        super().__init__()
        self.app = app
        self.current_block = 0
        self.validator = Validator(record_size, index_record_size)
        self.title("Lab 4 - Oleksandr Cherepov")
        self.geometry("800x500")

        style = ttk.Style()
        style.configure("Treeview.Heading", font=('Arial', 10, 'bold'))

        self._setup_ui()
        self.refresh_table()

    def _setup_ui(self):
        button_frame = tk.Frame(self, pady=15, bg="#f0f0f0")
        button_frame.pack(side=tk.TOP, fill=tk.X)

        btn_params = {"padx": 10, "pady": 5, "expand": True, "fill": tk.X}

        tk.Button(button_frame, text="Add Record",
command=self._on_add_click).pack(side=tk.LEFT, **btn_params)

        tk.Button(button_frame, text="Find Record",
command=self._on_find_click).pack(side=tk.LEFT, **btn_params)

        tk.Button(button_frame, text="Update Record",
command=self._on_update_click).pack(side=tk.LEFT, **btn_params)

```



```

        tk.Button(button_frame, text="Delete Record",
command=self._on_delete_click).pack(side=tk.LEFT, **btn_params)

        table_container = tk.Frame(self)
        table_container.pack(side=tk.TOP, fill=tk.BOTH, expand=True, padx=20,
pady=10)

        scrollbar = ttk.Scrollbar(table_container)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        self.tree = ttk.Treeview(
            table_container,
            columns=("Key", "Value", "Area"),
            show='headings',
            yscrollcommand=scrollbar.set
        )

        self.tree.heading("Key", text="Key")
        self.tree.heading("Value", text="Data Content")
        self.tree.heading("Area", text="Storage Area")

        self.tree.column("Key", width=80, anchor=tk.CENTER)
        self.tree.column("Value", width=450, anchor=tk.W)
        self.tree.column("Area", width=120, anchor=tk.CENTER)

        self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar.config(command=self.tree.yview)

        nav_frame = tk.Frame(self, pady=10)
        nav_frame.pack(side=tk.BOTTOM, fill=tk.X)

        self.btn_prev = tk.Button(nav_frame, text="◀", command=self._prev_block)
        self.btn_prev.pack(side=tk.LEFT, padx=20)

        self.btn_next = tk.Button(nav_frame, text="▶", command=self._next_block)
        self.btn_next.pack(side=tk.RIGHT, padx=20)

    def refresh_table(self, block=None):
        if block is None:
            block = self.current_block
        else:
            self.current_block = block

        for item in self.tree.get_children():
            self.tree.delete(item)

        records, total_blocks = self.app.get_records(block)

        for rec in records:
            self.tree.insert("", tk.END, values=(
                rec['key'],

```

```

        rec['value'],
        rec['area'].upper()
    ))

    self.btn_prev.config(state=tk.NORMAL if block > 0 else tk.DISABLED)
    self.btn_next.config(state=tk.NORMAL if block < total_blocks - 1 else
tk.DISABLED)

def _next_block(self):
    self.current_block += 1
    self.refresh_table()

def _prev_block(self):
    self.current_block -= 1
    self.refresh_table()

def _on_add_click(self):
    add_window = Toplevel(self)
    add_window.title("Add New Record")
    add_window.geometry("250x200")
    add_window.transient(self)
    add_window.grab_set()

    Label(add_window, text="Key (leave -1 for auto):").pack(pady=(10, 0))
    key_entry = Entry(add_window)
    key_entry.insert(0, "-1")
    key_entry.pack(padx=20, fill=X)

    Label(add_window, text="Data Value:").pack(pady=(10, 0))
    value_entry = Entry(add_window)
    value_entry.pack(padx=20, fill=X)

    def submit():
        key_val = key_entry.get()
        data_val = value_entry.get()

        key_validation, msg = self.validator.validate_key(key_val)

        if not key_validation:
            messagebox.showwarning(msg[0], msg[1])
            return

        data_validation, msg = self.validator.validate_data(data_val)

        if not data_validation and key_validation:
            messagebox.showwarning(msg[0], msg[1])
            return

        new_id = self.app.add(data_val, int(key_val))

        if new_id != -1:

```

```

        messagebox.showinfo("Success", f"Record added successfully with
ID: {new_id}")
        add_window.destroy()
        self.refresh_table()
    else:
        messagebox.showerror("Error", "Failed to add record. ID might
already exist.")

    Button(add_window, text="Add Record", command=submit).pack(pady=20)

def _on_find_click(self):
    key_to_find = simpledialog.askinteger("Find Record", "Enter Key to
search:")

    if key_to_find is None:
        return

    result = self.app.search(key_to_find)

    if result != -1:
        msg = (f"Key: {result['key']}\n"
              f"Value: {result['value']}\n"
              f"Area: {result['area']}")
        messagebox.showinfo("Record Found", msg)

        target_page = self._get_page_for_record(result['key'])

        if target_page != -1: self.refresh_table(target_page)

        self._highlight_row_by_key(result['key'])
    else:
        messagebox.showwarning("Not Found", f"Record with key {key_to_find}
not found!")

def _highlight_row_by_key(self, key):
    for item_id in self.tree.get_children():
        row_key = self.tree.item(item_id)['values'][0]

        if str(row_key) == str(key):
            self.tree.selection_set(item_id)
            self.tree.see(item_id)
            return

def _get_page_for_record(self, key):
    all_indices = self.app.storage.get_all_records()

    all_indices.sort(key=lambda x: x[0])

    for global_index, (k, _) in enumerate(all_indices):
        if k == key:
            return global_index // self.app.PAGE_LIMIT

```

```

        return -1

def _on_update_click(self):
    selected = self.tree.selection()
    if not selected:
        messagebox.showwarning("Warning", "Please select a record to delete")
        return

    item_id = selected[0]
    row_values = self.tree.item(item_id)['values']
    current_key = row_values[0]
    current_val = row_values[1]

    update_win = Toplevel(self)
    update_win.title(f"Update Record #{current_key}")
    update_win.geometry("400x200")
    update_win.transient(self)
    update_win.grab_set()

    Label(update_win, text=f"Key: {current_key}", font=('Arial', 10,
'bold')).pack(pady=10)

    Label(update_win, text="Enter New Data Value").pack()
    val_entry = Entry(update_win)
    val_entry.insert(0, current_val)
    val_entry.pack(padx=20, fill=X)

def do_save():
    new_data = val_entry.get()

    data_validation, msg = self.validator.validate_data(new_data)

    if not data_validation:
        messagebox.showwarning(msg[0], msg[1])
        return

    try:
        result = self.app.update(int(current_key), new_data)

        if result != -1:
            messagebox.showinfo("Success", "Record updated successfully!")
            update_win.destroy()
            self.refresh_table()
        else:
            messagebox.showerror("Error", "Failed to update record in
storage.")
    except Exception as e:
        messagebox.showerror("Error", f"An unexpected error occurred:
{e}")

```

```

        Button(update_win, text="Save Changes", command=do_save).pack(pady=20)

def _on_delete_click(self):
    selected = self.tree.selection()
    if not selected:
        messagebox.showwarning("Warning", "Please select a record to delete")
        return

    item_data = self.tree.item(selected[0])
    key_to_delete = item_data['values'][0]

    if messagebox.askyesno("Confirm", f"Are you sure you want to delete this
record? (key: {key_to_delete})"):
        self.app.remove(key_to_delete)
        self.refresh_table()

```

### ***logger.py:***

```

import logging
import sys

class Logger(logging.Logger):
    def __init__(self, name):
        super().__init__(name)
        handler = logging.StreamHandler()
        formatter = logging.Formatter('%(name)s - %(levelname)s - %(message)s')
        handler.setFormatter(formatter)
        self.addHandler(handler)

    def error_and_exit(self, message, status):
        self.error(message)
        self.info(f"Exiting with {status}...")
        sys.exit(status)

```

### ***validator.py:***

```

class Validator:
    def __init__(self, record_size, index_record_size):
        self.RECORD_SIZE = record_size
        self.INDEX_RECORD_SIZE = index_record_size

    def validate_key(self, key_value):
        if '.' in key_value or ',' in key_value:
            return False, ("Input Error", "Index must be an integer!")

```



```
root.mainloop()
```

### 3.3.2 Приклади роботи

На рисунках 3.1 і 3.2 показані приклади роботи програми для додавання і пошуку запису.

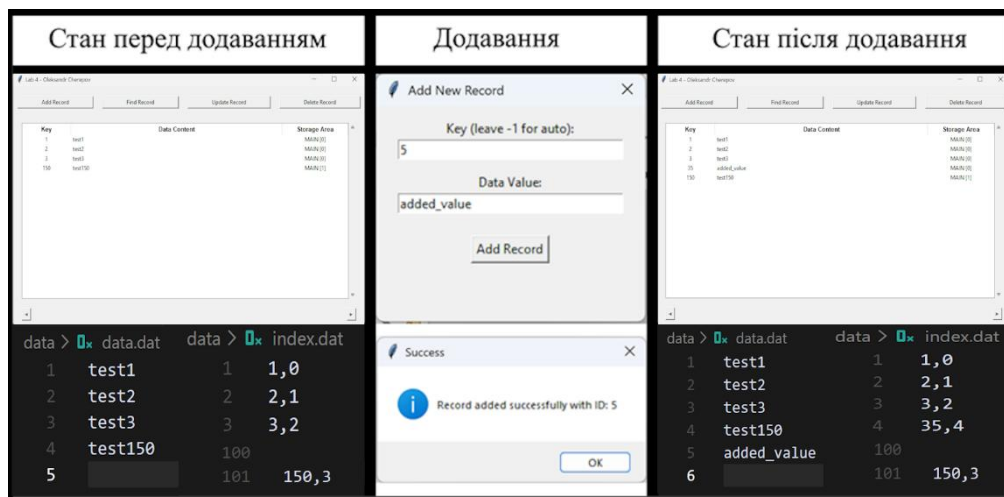


Рисунок 3.1 – Додавання запису

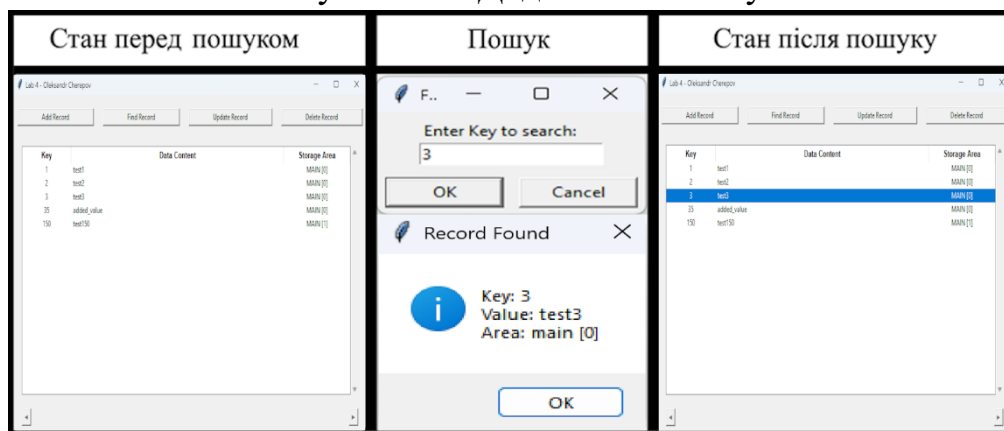


Рисунок 3.2 – Пошук запису

### 3.4 Тестування алгоритму

#### 3.4.1 Часові характеристики оцінювання

В таблиці 3.1 наведено кількість порівнянь для 20 спроб пошуку запису по ключу.

Таблиця 3.1 – Число порівнянь при спробі пошуку запису по ключу

| Номер спроби пошуку ( $i$ ) | Число порівнянь ( $c_i$ ) |
|-----------------------------|---------------------------|
| 1                           | 4                         |
| 2                           | 2216                      |
| 3                           | 6                         |
| 4                           | 3276                      |
| 5                           | 4                         |
| 6                           | 4                         |
| 7                           | 4523                      |
| 8                           | 10                        |
| 9                           | 481                       |
| 10                          | 8                         |
| 11                          | 2700                      |
| 12                          | 6                         |
| 13                          | 624                       |
| 14                          | 1647                      |
| 15                          | 4                         |
| 16                          | 2140                      |
| 17                          | 2075                      |
| 18                          | 3745                      |
| 19                          | 598                       |
| 20                          | 2811                      |

Середня кількість порівнянь:  $\frac{\sum_{i=1}^{20} c_i}{20} = \frac{26882}{20} = 1344,1 \approx 1344$ .



## ВИСНОВОК

В рамках лабораторної роботи було вивчено підходи до проєктування складних структур даних та реалізовано СУБД на основі файлів із щільним індексом та областю переповнення. Основний пошук впорядкованих ключів реалізовано методом Шарра, що забезпечує високу швидкість доступу до записів.

Експериментальне тестування на базі з 5 000 записів зафіксувало середнє число порівнянь 1344. Таке значення формується на основі великої кількості запитів пошуку до області переповнення, де використовується лінійний пошук. Якщо ж брати до уваги виключно запити до основної області, то середня кількість порівнянь приблизно дорівнює 6. Це демонструє значну перевагу інтерполяційного підходу над лінійним перебором.

Чат-бот на основі моделі Claude Sonnet 4.5 (Free plan) дав наступні рекомендації щодо покращення програми, зокрема алгоритму пошуку:

- Технічна оптимізація: впровадити кешування індексів у пам'яті та перейти на текстовий режим читання файлів для зниження навантаження на диск.
- Алгоритмічне покращення: замінити лінійний пошук в області переповнення на бінарний та використовувати *enumerate* для прискорення ітерацій.
- Архітектурний рефакторинг: використовувати *dataclass* для типізації результатів пошуку та винести логіку валідації в окремі модулі для підвищення надійності коду.

## КРИТЕРІЇ ОЦІНЮВАННЯ

Максимальний бал за лабораторну роботу №4 дорівнює – 10.

Критерії оцінювання:

- проектування алгоритму – 1;
- програмна реалізація алгоритму – 2;
- захист – 4;
- дослідження алгоритму – 2;
- звіт – 1.